

Sumario

1. Introducción.....	2
1.1 Ejemplo de programación de Sockets.....	5
1.1.1 Actividades.....	8
2. Servidor multiciente.....	9
2.1 Actividades.....	15
3. Sockets, ampliando información.....	16
4.1 Mapa conceptual.....	5
5. Ejemplo.....	6
6. Actividad 1.....	13
7. Actividad 2.....	14
8. Sockets e hilos.....	15
8.1 Actividades.....	21

1. Introducción

La programación de **sockets** en Java se refiere a la creación de aplicaciones que utilizan sockets para la comunicación en red. Los sockets son una API estándar que permite que dos aplicaciones en diferentes máquinas o en la misma máquina se comuniquen entre sí a través de una red. Los sockets proporcionan una forma estándar y eficiente de enviar y recibir datos entre procesos en diferentes dispositivos.

Un socket representa una combinación de una **dirección IP** y un **número de puerto** que identifica de manera única una comunicación entre dos dispositivos en una red.

Los sockets se utilizan ampliamente en aplicaciones que requieren comunicación en red, como navegadores web (que utilizan sockets para solicitar y recibir páginas web), servicios de correo electrónico, juegos en línea, transferencia de archivos y muchas otras aplicaciones distribuidas.

Los sockets utilizan un puerto acordado entre el cliente y el servidor, existente en el servidor.

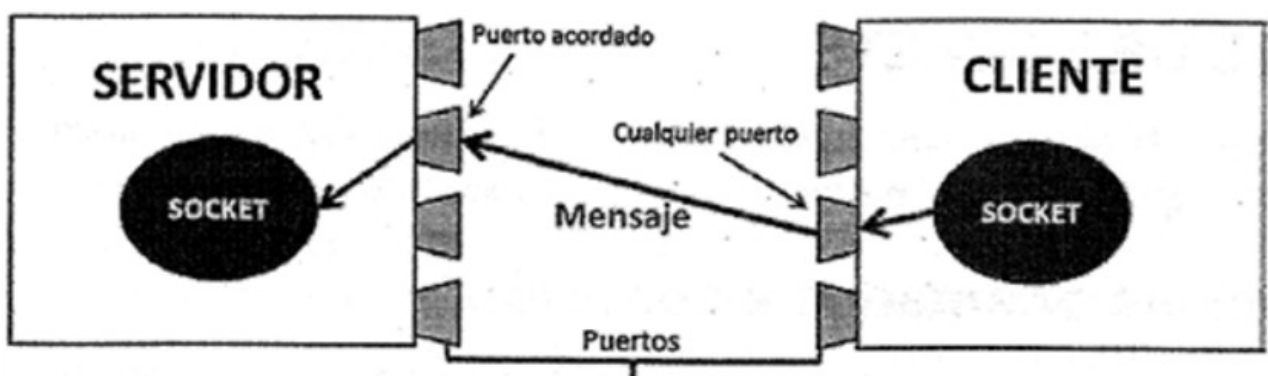
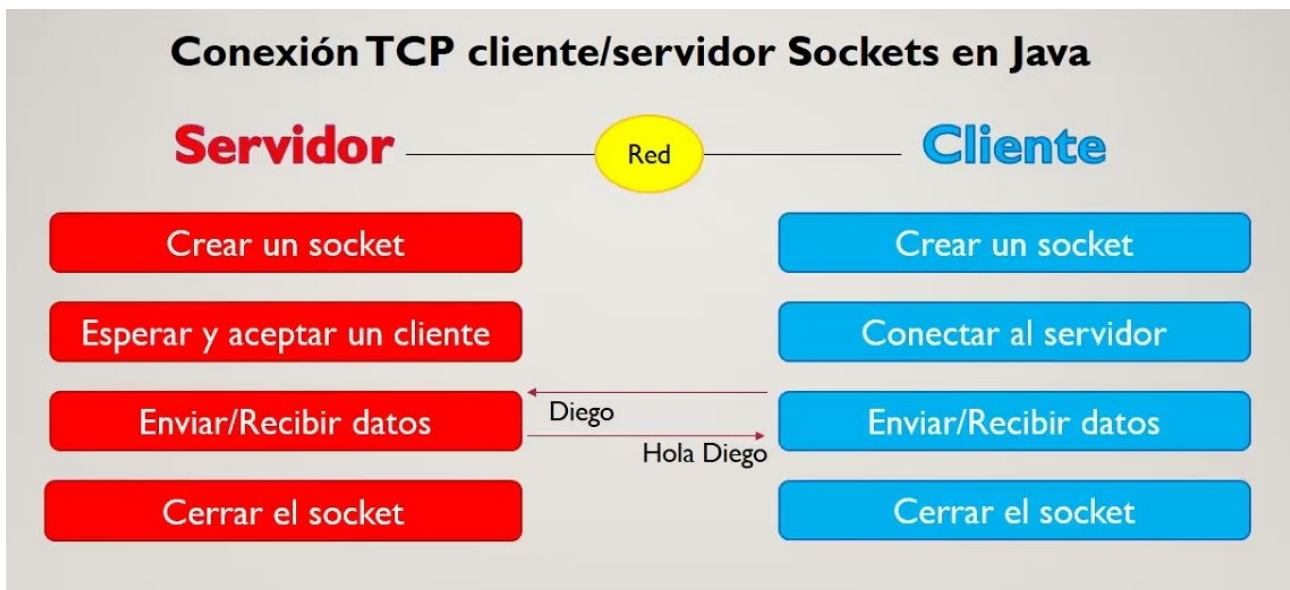
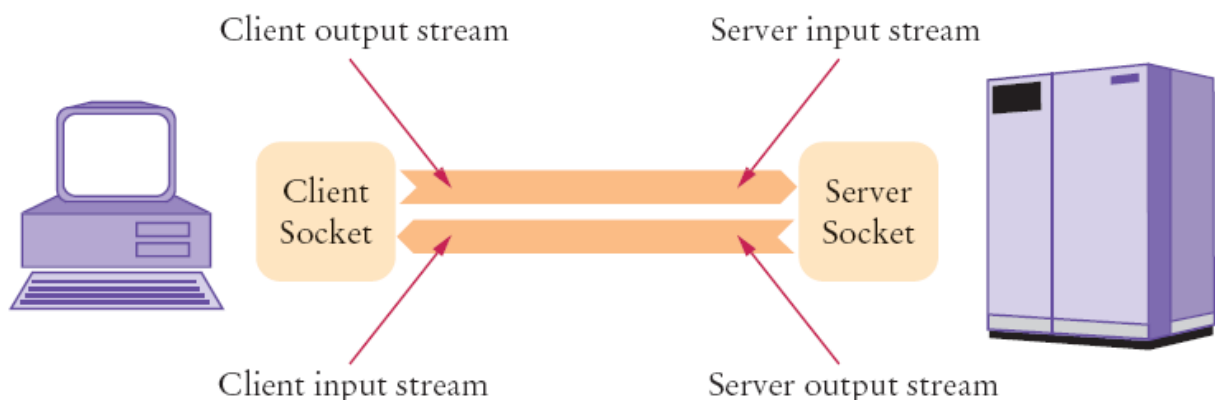


Figura 3.2. Socket y puertos.

En la imagen siguiente quedan representados los diferentes pasos en la comunicación entre un servidor y un cliente mediante el empleo de Sockets:



Para que pueda existir comunicación entre un cliente y un servidor el Socket establecerá dos vías de comunicación, en cada una de ellas transferirán los bytes de información en un único sentido: Desde el cliente al servidor (el cliente emite información mientras el servidor la recibe) o bien, desde el servidor al cliente (el servidor emitirá bytes de información mientras el cliente únicamente los recibe):



Para crear un socket de cliente en Java, puedes utilizar la clase `Socket`. Para crear un socket de servidor en Java, puedes utilizar la clase `ServerSocket`. Ambas clases se encuentran respectivamente en las librerías: `java.net.ServerSocket` y `java.net.Socket`, ambas librerías deberán ser importadas.

1.1 Ejemplo de programación de Sockets

A continuación vamos a ver como dos aplicaciones programadas en Java se comunican entre sí en un mismo ordenador (localhost). Una de ellas (Server.java) actuará como servidor, la otra (Client.java) lo hará como cliente. El Servidor iniciará su ejecución esperando recibir un mensaje del cliente (“Buenos días soy un cliente”), en el momento de la recepción de este mensaje, el servidor contestará al cliente: “Hola amigo, qué quieres”.

Ambas aplicaciones se ejecutan desde el mismo equipo, por lo tanto deberás abrir dos terminales y en cada uno de ellos generar inicialmente el archivo .class correspondiente al .java:

```
javac sockets1/Server.java sockets1/Client.java
```

Finalmente desde cada terminal iniciar la ejecución de ambos procesos:

```
java sockets1.Server
```

```
java sockets1.Client
```

El código de **Server.java** y de **Client.java** es el siguiente:

```
package sockets1;

import java.io.*;
import java.net.*;

public class Server {

    public static void main(String[] args) {

        int puerto = 5000;

        try (ServerSocket serverSocket = new ServerSocket(puerto)) {

            System.out.println("Servidor esperando conexión...");

            Socket socket = serverSocket.accept();

            System.out.println("Cliente conectado");

            BufferedReader entrada = new BufferedReader(

                new InputStreamReader(socket.getInputStream()));

            PrintWriter salida = new PrintWriter(

                socket.getOutputStream(), true);

            // Recibir mensaje del cliente

            String mensaje = entrada.readLine();

            System.out.println("Mensaje recibido: " + mensaje);

            // Responder al cliente

            salida.println("Hola amigo, qué quieres ?");

            socket.close();

            System.out.println("Conexión cerrada");

        } catch (IOException e) {

            e.printStackTrace();        }    }}
```

Client.java

```
package sockets1;

import java.io.*;

import java.net.*;

public class Client {

    public static void main(String[] args) {

        String host = "localhost"; // mismo equipo

        int puerto = 5000;

        try (Socket socket = new Socket(host, puerto)) {

            BufferedReader entrada = new BufferedReader(

                new InputStreamReader(socket.getInputStream()));

            PrintWriter salida = new PrintWriter(

                socket.getOutputStream(), true);

            // Enviar mensaje al servidor

            salida.println("Buenos días soy un cliente ");

            // Recibir respuesta del servidor

            String respuesta = entrada.readLine();

            System.out.println("Respuesta del servidor: " + respuesta);

        } catch (IOException e) {

            e.printStackTrace();

        }

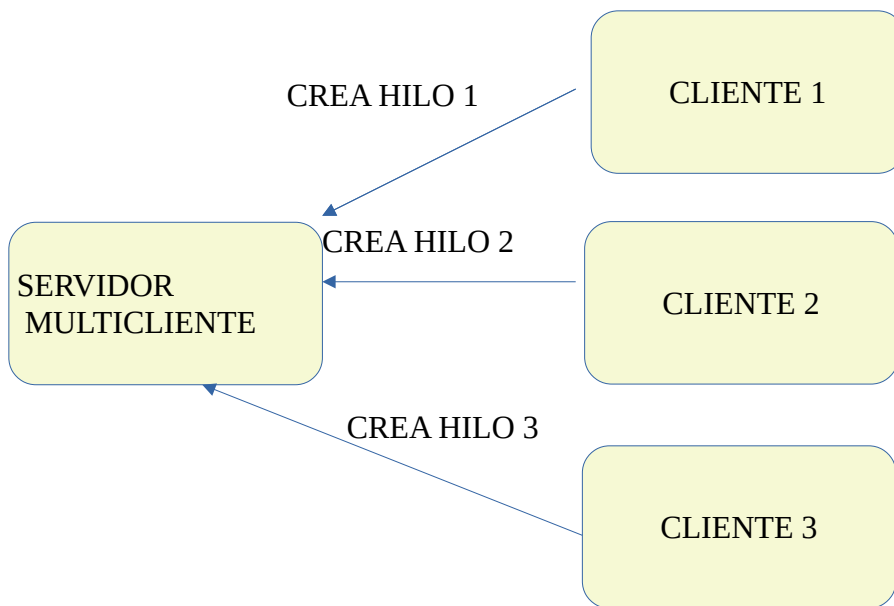
    }

}
```

1.1.1 Actividades

1. Implementa y ejecuta el programa(cada uno en un terminal diferentes) anterior (Server.java y Client.java).
2. Prueba ejecutar estos dos procesos en equipos diferentes del aula (en lugar de indicar localhost tendrás que especificar la ip de cada equipo).
3. Realiza un programa que vaya generando 10 números aleatorios y los vaya enviando a otro programa encargado de calcular la suma de todos los números recibidos y devolverle el resultado de dicha suma multiplicado por 10.
4. Vas a simular el funcionamiento de un chat entre dos procesos

2. Servidor multicliente



Un **servidor multicliente** es una aplicación diseñada para **atender a varios clientes al mismo tiempo**, gestionando múltiples conexiones de forma simultánea. Por cada conexión el **SERVIDOR** creará un hilo de ejecución de forma que en ese nuevo hilo, el cliente podrá ir haciendo tantas peticiones (en este ejemplo solicitud de comandos) como quiera. Cada uno de los hilos creados por el servidor y asociados a un único cliente podrá ir dando respuestas a cada uno de los comandos o ordenes de su respectivo **CLIENTE**.

Servidor muticliente:

```
import java.io.*;

import java.net.*;

import java.time.LocalDateTime;

public class ServidorServicios {

    public static void main(String[] args) {

        int puerto = 5000;

        System.out.println("Servidor iniciado en el puerto " + puerto);

        try (ServerSocket serverSocket = new ServerSocket(puerto)) {

            while (true) {

                Socket cliente = serverSocket.accept();

                new Thread(new ManejadorCliente(cliente)).start();

            }

        } catch (IOException e) {

            e.printStackTrace();

        }

    }

}
```

```

public class ManejadorCliente implements Runnable {

    private Socket socket;

    public ManejadorCliente(Socket socket) {

        this.socket = socket;
    }

    @Override

    public void run() {

        try (

            BufferedReader entrada = new BufferedReader(

                new InputStreamReader(socket.getInputStream()));

            PrintWriter salida = new PrintWriter(

                socket.getOutputStream(), true)

        ) {

            salida.println("Bienvenido. Comandos: HORA, SUMA a b, SALIR");

            String mensaje;

            while ((mensaje = entrada.readLine()) != null) {

                if (mensaje.equalsIgnoreCase("SALIR")) {

                    salida.println("Conexión cerrada");

                    break;

                }

                else if (mensaje.equalsIgnoreCase("HORA")) {

                    salida.println("Hora del servidor: " + LocalTime.now());

                }

                else if (mensaje.startsWith("SUMA")) {

                    String[] partes = mensaje.split(" ");

```

```
        if (partes.length == 3) {

            int a = Integer.parseInt(partes[1]);

            int b = Integer.parseInt(partes[2]);

            salida.println("Resultado: " + (a + b));

        } else {

            salida.println("Uso INCORRECTO: SUMA a b");

        }

    }

    else {

        salida.println("Comando no reconocido");

    }

}

socket.close();

} catch (Exception e) {

    System.out.println("Cliente desconectado");

}

}

}
```

Cliente:

```
import java.io.*;

import java.net.*;

public class ClienteServicios {

    public static void main(String[] args) {

        String host = "localhost";

        int puerto = 5000;

        try (

            Socket socket = new Socket(host, puerto);

            BufferedReader teclado = new BufferedReader(

                new InputStreamReader(System.in));

            BufferedReader entrada = new BufferedReader(

                new InputStreamReader(socket.getInputStream()));

            PrintWriter salida = new PrintWriter(

                socket.getOutputStream(), true)

        ) {

            System.out.println(entrada.readLine());

            String comando;

            while (true) {

                System.out.print("> ");

                comando = teclado.readLine();

                salida.println(comando);

                String respuesta = entrada.readLine();

                System.out.println(respuesta);

                if (comando.equalsIgnoreCase("SALIR")) {
```

```
        break;

    }

}

} catch (Exception e) {

    e.printStackTrace();

}

}

}
```

2.1 Actividades

1. Prueba ejecutar las dos clases anteriores (servidor muticliente y cliente) en dos terminales de tu equipo.
2. Prueba ejecutar la clase servidorMulticliente en un terminal, y dos clases cliente en dos terminales, todas de tu mismo equipo.
3. Prueba ejecutar la clase servidorMulticliente en un terminal de tu equipo y la clase cliente en otro terminal de otro equipo del aula.

3. Sockets, ampliando información

Los sockets son una interfaz de programación que permite la comunicación entre aplicaciones en una red, ya sea a través de una red local (LAN) o a través de Internet. Los sockets proporcionan una forma estándar de enviar y recibir datos entre dos nodos en una red. Existen varios tipos de sockets, pero los más comunes son los sockets de flujo (stream sockets) y los sockets de datagramas (datagram sockets).

1. Sockets de Flujo (Stream Sockets):

- También conocidos como sockets TCP (Transmission Control Protocol).
- Proporcionan una comunicación orientada a la conexión y confiable.
- Los datos se envían en un flujo continuo, lo que garantiza que se entreguen en orden y sin pérdida.
- Se utilizan comúnmente en aplicaciones que requieren una transferencia confiable de datos, como la transmisión de archivos, navegadores web y correo electrónico.

2. Sockets de Datagramas (Datagram Sockets):

- También conocidos como sockets UDP (User Datagram Protocol).
- Proporcionan una comunicación sin conexión y no confiable.
- Los datos se envían en paquetes independientes, lo que significa que no se garantiza la entrega en orden ni sin pérdida.
- Se utilizan en aplicaciones donde la velocidad y la eficiencia son más importantes que la confiabilidad, como videoconferencias, juegos en línea y aplicaciones de transmisión en tiempo real.

3. Sockets Crudos (Raw Sockets):

- Permiten a las aplicaciones acceder a niveles más bajos de la pila de red.

- Se utilizan para realizar operaciones avanzadas, como la construcción de paquetes personalizados o el análisis de tráfico de red.
- Requieren permisos especiales y generalmente están disponibles solo para administradores del sistema.

4. Sockets Secos (Socket SOCKS):

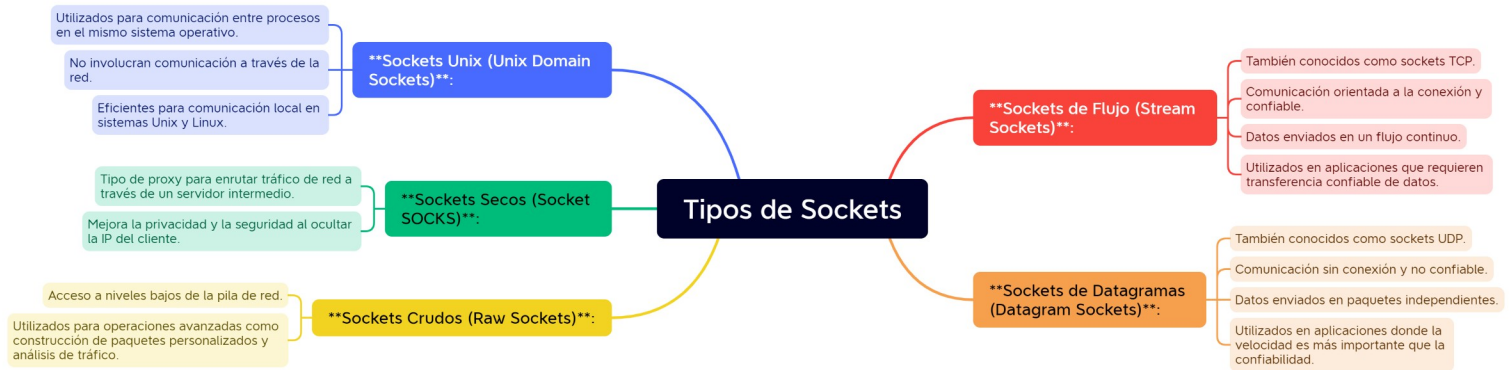
- Son un tipo de proxy utilizado para enrutar el tráfico de red a través de un servidor intermedio.
- Se utilizan para mejorar la privacidad y la seguridad al ocultar la dirección IP del cliente.
- Comúnmente utilizados en aplicaciones que requieren acceso a Internet anónimo, como navegadores web o aplicaciones de torrent.

5. Sockets Unix (Unix Domain Sockets):

- Se utilizan para la comunicación entre procesos en el mismo sistema operativo.
- No involucran la comunicación a través de una red, lo que los hace más eficientes que los sockets de red para la comunicación local.
- Se utilizan en sistemas Unix y variantes como Linux.

Cada tipo de socket tiene su propósito y se adapta a diferentes necesidades de comunicación en una red. La elección del tipo de socket depende de los requisitos de la aplicación y de si se necesita una comunicación confiable, rápida o anónima, entre otros factores.

4.1 Mapa conceptual



Presented with xmind

5. Ejemplo

A continuación se puede ver el código java de un programa **servidor** que hace uso de Sockets:

```
import java.io.*;
import java.net.ServerSocket;
import java.net.Socket;

public class ServidorSocket {

    public static void main(String[] args) {

        final int puerto = 12348; // Puerto en el que el servidor escuchará

        try (ServerSocket servidorSocket = new ServerSocket(puerto)) {

            System.out.println("Servidor esperando conexiones en el puerto " +
puerto);

            // Aceptar la conexión del cliente

            Socket clienteSocket = servidorSocket.accept();

            System.out.println("Cliente conectado desde " +
clienteSocket.getInetAddress());
```

```
// Obtener flujos de entrada y salida

    PrintWriter out = new PrintWriter(clienteSocket.getOutputStream(),
true);

    BufferedReader in = new BufferedReader(new
InputStreamReader(clienteSocket.getInputStream()));

    // Leer el mensaje del cliente y mostrarlo

    String mensajeCliente = in.readLine();

    System.out.println("Mensaje del cliente: " + mensajeCliente);

    // Enviar una respuesta al cliente

    out.println("Mensaje recibido por el servidor: " + mensajeCliente);

    // Cerrar la conexión

    clienteSocket.close();
} catch (IOException e) {
    e.printStackTrace();
}
}
```

Para crear un socket de cliente en Java, puedes utilizar la **clase** `Socket`. Seguidamente se puede ver el código Java del **cliente** como emisor inicial del mensaje:

```
import java.io.*;
import java.net.ServerSocket;
import java.net.Socket;

public class ServidorSocket {

    public static void main(String[] args) {

        final int puerto = 12348; // Puerto en el que el servidor escuchará

        try (ServerSocket servidorSocket = new ServerSocket(puerto)) {

            System.out.println("Servidor esperando conexiones en el puerto " + puerto);

            // Aceptar la conexión del cliente

            Socket clienteSocket = servidorSocket.accept();

            System.out.println("Cliente conectado desde " + clienteSocket.getInetAddress());

            // Obtener flujos de entrada y salida

            PrintWriter out = new PrintWriter(clienteSocket.getOutputStream(), true);

            BufferedReader in = new BufferedReader(new InputStreamReader(clienteSocket.getInputStream()));

            // Leer el mensaje del cliente y mostrarlo

            String mensajeCliente = in.readLine();

            System.out.println("Mensaje del cliente: " + mensajeCliente);

            // Enviar una respuesta al cliente

            out.println("Mensaje recibido por el servidor: " + mensajeCliente);

            // Cerrar la conexión

            clienteSocket.close();

        } catch (IOException e) {

            e.printStackTrace();

        }
    }
}
```

A continuación se van a modificar los programas anteriores (el del servidor y el del cliente) de forma que el cliente solicitará al usuario la introducción de un número que será enviado al servidor para que lo incremente en 10 unidades y lo vuelva a enviar al cliente que mostrará su nuevo valor en la consola. El código de los programas anteriores que ha sido modificado que marcado en color rojo:

Servidor

```
import java.io.*;

import java.net.ServerSocket;

//import java.net.*;

import java.net.Socket;

public class ServidorSocket {

    public static void main(String[] args) {

        final int puerto = 12348; // Puerto en el que el servidor escuchará

        int result;

        try (ServerSocket servidorSocket = new ServerSocket(puerto)) {

            System.out.println("Servidor esperando conexiones en el puerto " + puerto);

            // Aceptar la conexión del cliente

            Socket clienteSocket = servidorSocket.accept();

            System.out.println("Cliente conectado desde " + clienteSocket.getInetAddress());
```

```
// Obtener flujos de entrada y salida

    PrintWriter out = new PrintWriter(clienteSocket.getOutputStream(), true);

    BufferedReader in = new BufferedReader(new
InputStreamReader(clienteSocket.getInputStream()));

    // Leer el mensaje del cliente y mostrarlo

    String mensajeCliente = in.readLine();

    result=Integer.parseInt(mensajeCliente);

    result=result+10;

    // Enviar una respuesta al cliente

    mensajeCliente=String.valueOf(result);

    out.println(mensajeCliente);

    // Cerrar la conexión

    clienteSocket.close();

} catch (IOException e) {

    e.printStackTrace();

}

}

}
```

Cliente

```
import java.io.*;

import java.net.Socket;

//import java.net.*;

import java.util.Scanner;

public class ClienteSocket {

    public static void main(String[] args) {

        Scanner sc=new Scanner(System.in);

        final String host = "192.168.1.137"; // Dirección del servidor

        final int puerto = 12348; // Puerto del servidor

        try (Socket clienteSocket = new Socket(host, puerto);

            PrintWriter out = new PrintWriter(clienteSocket.getOutputStream(), true);

            BufferedReader in = new BufferedReader(new

InputStreamReader(clienteSocket.getInputStream())) {

            System.out.println("Conectado al servidor en " + host + ":" + puerto);
```

```
// Enviar un mensaje al servidor

System.out.println("Escribe un número para que el servidor lo incremente en 10
unidades");

String numero=sc.nextLine();

out.println(numero);

// Leer la respuesta del servidor y mostrarla

String respuestaServidor = in.readLine();

System.out.println("Respuesta del servidor: el número incrementado en 10 unidades es"
+ respuestaServidor);

    } catch (IOException e) {
        e.printStackTrace();
    }
}
}
```

6. Actividad 1

1. Vas a probar el funcionamiento del ejemplo anterior de las formas siguientes:

- Debes de ejecutar tanto el cliente como el servidor, en la consola de *Eclipse*, desde tu ordenador.
- Ejecuta el programa servidor desde un ordenador de clase y, el programa cliente desde otro ordenador de clase.
- Ejecuta el programa servidor en tu equipo con tu sistema operativo principal mientras que el programa cliente lo ejecutarás desde una máquina virtual ubicada en tu equipo.

7. Actividad 2

1. **Ejercicio de Echo Server-Cliente:** Crea un servidor que escuche en un puerto específico y un cliente que se conecte a ese servidor. El cliente debe enviar un mensaje al servidor, y el servidor debe devolver el mismo mensaje de vuelta al cliente.
2. **Ejercicio de Chat Simple:** Implementa un chat simple donde un servidor puede recibir mensajes de varios clientes y reenviar esos mensajes a todos los clientes conectados. Cada cliente debe ser capaz de enviar mensajes al servidor, y el servidor los distribuirá a todos los demás clientes.
3. **Ejercicio de Adivinanza:** Implementa un juego de adivinanza en el que un cliente adivina un número entre 1 y 100 que el servidor ha elegido al azar. El servidor debe dar pistas al cliente si el número es mayor o menor hasta que el cliente adivine correctamente.
4. **Ejercicio de Consulta de Diccionario:** Crea un servidor que actúe como un diccionario simple. El cliente debe enviar una palabra al servidor, y el servidor debe responder con la definición de esa palabra si está en su "diccionario" interno.
5. **Ejercicio de Autenticación de Usuario:** Implementa un sistema de autenticación en el que el cliente envía un nombre de usuario y una contraseña al servidor para iniciar sesión. El servidor debe verificar las credenciales y responder si el inicio de sesión fue exitoso o no.

8. Sockets e hilos

La combinación de sockets y hilos en Java es una técnica común para crear aplicaciones de red que pueden manejar múltiples clientes de manera concurrente. Los sockets se utilizan para la comunicación entre el servidor y los clientes, mientras que los hilos se utilizan para manejar cada cliente de forma independiente, lo que permite que varios clientes se conecten y realicen operaciones al mismo tiempo.

A continuación se puede ver el código Java de un servidor que permanece a la escucha de las peticiones de los clientes. Por cada solicitud de un cliente, el servidor generará un hilo para gestionar y dar respuesta al cliente:

```
import java.io.*;
import java.net.*;
import java.util.concurrent.*;

public class socketshilos {

    public static void main(String[] args) {

        int portNumber = 12345;

        ServerSocket serverSocket = null;

        try {

            serverSocket = new ServerSocket(portNumber);

            System.out.println("Servidor escuchando en el puerto " + portNumber);

            while (true) {

                Socket clientSocket = serverSocket.accept();

                System.out.println("Cliente conectado desde " + clientSocket.getInetAddress());
```

```
// Creamos un nuevo hilo para manejar al cliente

Thread clientThread = new Thread(new ClientHandler(clientSocket));

clientThread.start();

}

} catch (IOException e) {

    System.err.println("Error en el servidor: " + e.getMessage());

} finally {

    try {

        if (serverSocket != null) {

            serverSocket.close();

        }

    } catch (IOException e) {

        System.err.println("Error al cerrar el servidor: " + e.getMessage());

    }

}

}
```

```
public class ClientHandler implements Runnable {

    private Socket clientSocket;

    public ClientHandler(Socket socket) {

        this.clientSocket = socket;

    }

    public void run() {

        //      System.out.println("aaaaaaaaaaaaaaaaaaaaa");

        try (

            PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);

            BufferedReader in = new BufferedReader(new

InputStreamReader(clientSocket.getInputStream()));

        ) {

            String inputLine;

            while ((inputLine = in.readLine()) != null) {

                System.out.println("Mensaje del cliente: " + inputLine);

                // Enviar una respuesta al cliente

                out.println("Hola, cliente. Has enviado: " + inputLine);

            }

        } catch (IOException e) {

            System.err.println("Error en la comunicación con el cliente: " + e.getMessage());

        }

    }

}
```

```
finally {  
    try {  
        clientSocket.close();  
    } catch (IOException e) {  
        System.err.println("Error al cerrar el socket del cliente: " + e.getMessage());  
    }  
}  
}  
}
```

Por el otro lado, se podrían ejecutar tantos programas clientes como se quisieran. En el ejemplo siguiente se va a ejecutar un único programa cliente aunque generará 10 hilos de ejecución cada uno de los cuales enviará un mensaje al servidor indicando cual es su identificador. Este, el servidor, dará respuesta (mediante el correspondiente hilo) al hilo cliente:

```
import java.io.*;
import java.net.*;

public class clientesHilos {
    public static void main(String[] args) {
        int numClients = 10; // Número de clientes a crear
        for (int i = 0; i < numClients; i++) {
            Thread clientThread = new Thread(new ClientRunnable(i));
            clientThread.start();
        }
    }
}
```

```
public class ClientRunnable implements Runnable {  
    private int clientId;  
    public ClientRunnable(int id) {  
        this.clientId = id;  
    }  
    public void run() {  
        String serverAddress = "localhost"; // Cambia a la dirección IP o nombre del  
servidor si es diferente  
        int serverPort = 12345; // Asegúrate de usar el mismo puerto que el servidor  
        try (Socket socket = new Socket(serverAddress, serverPort);  
            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);  
            BufferedReader in = new BufferedReader(new  
InputStreamReader(socket.getInputStream())) {  
            // Envía un mensaje al servidor desde este cliente  
            String message = "Hola, servidor. Soy el cliente " + clientId;  
            out.println(message);  
            // Recibe la respuesta del servidor  
            String response = in.readLine();  
            System.out.println("Respuesta del servidor para el cliente " + clientId  
+ ": " + response);  
        } catch (UnknownHostException e) {  
            System.err.println("No se puede encontrar el host: " + serverAddress);  
        } catch (IOException e) {  
            System.err.println("Error de E/S en el cliente " + clientId + ": " +  
e.getMessage());  
        }  
    }  
}
```

8.1 Actividades

1. Eco Server y Cliente:

- Crea un servidor que escuche en un puerto y un cliente que se conecte a ese puerto.
- El servidor debe recibir un mensaje del cliente y devolver el mismo mensaje (eco) al cliente.
- Utiliza hilos para manejar múltiples conexiones de clientes.

2. Calculadora Simple:

- Diseña un servidor que acepte solicitudes de cálculos matemáticos básicos (suma, resta, multiplicación, división).
- Los clientes envían solicitudes al servidor, que realiza los cálculos y devuelve el resultado.
- Usa hilos para manejar las solicitudes de cálculo de cada cliente.

3. Juego de Preguntas y Respuestas:

- Crea un juego de preguntas y respuestas en el que el servidor envía preguntas a los clientes y recibe respuestas.
- El servidor debe llevar un registro de las puntuaciones de los clientes.
- Utiliza hilos para gestionar las sesiones de juego de cada cliente.

4. **Chat en tiempo real:** Diseña un programa de chat en tiempo real en el que varios clientes puedan conectarse al servidor y comunicarse entre ellos. Utiliza hilos para manejar a cada cliente de forma independiente. Los clientes deben poder enviar y recibir mensajes en tiempo real.
5. **Transferencia de archivos:** Crea un servidor que permita a los clientes cargar y descargar archivos. Los clientes deben poder enviar archivos al servidor y descargar archivos de otros clientes. Utiliza hilos para garantizar que las transferencias de archivos no bloqueen a otros clientes.
6. **Juego multijugador:** Implementa un juego multijugador simple, como el juego de "Adivina el número" o "Piedra, papel o tijeras". Los jugadores deben poder unirse al juego y competir entre ellos. Utiliza hilos para gestionar las sesiones de juego de cada jugador.